Segunda pre-entrega: ARM Assembler–Sokoban

Alumnos: Acuña Maximo y Casellas Santiago

Comisión: 3

Grupo: 27

Año: 2026

Pseudocódigo

```
inicio
	nivel <- elegir_nivel()
	cargar_mapa(nivel)
	movimientos <- 0
	t_inicio <- obtener_tiempo()
	salir <- falso
	mientras (no gano) y (no salir) hacer
		mostrar_mapa()
		tecla <- pedir_movimiento()
		si tecla = 'R' entonces
			cargar_mapa(nivel)
			movimientos <- 0
			t_inicio <- obtener_tiempo()
		sino si tecla = 'Q' entonces
			salir <- verdadero
		sino
			direccion <- obtener_posicion_siguiente(tecla)
			mov_valido <- verificar_y_mover(direccion)
			    si mov_valido = verdadero entonces
				movimientos <- movimientos + 1
		fin si
		gano <- verificar_victoria()
	fin mientras
	si gano entonces
		tiempo_total <- calcular_tiempo_de_juego(“Tiempo final”,
	t_inicio)
		escribir(“¡Has ganado! Ingresa tu nombre para guardar tu
	tiempo:”)
		leer(nombre)
		escribir_ranking(nombre, tiempo_total, movimientos)

	fin si
fin
```

DETALLES DE LA SUBRUTINA	CÓDIGO
Nombre: cargar_mapa Recibe: R0 (Dirección base del mapa original en .data) Devuelve: R0 (Desplazamiento inicial del personaje) Llama a: Ninguna	subrutina cargar_mapa(direccion_mapa_original) guardar_pila(LR) desplazamiento <- 0 posicion_personaje <- 0 mientras (caracter_actual != '\0') hacer caracter_actual <- leer_memoria(direccion_mapa_original + desplazamiento) escribir_memoria(direccion_matriz_activa + desplazamiento, caracter_actual)

	si caracter_actual = '@' o caracter_actual = '+' entonces posicion_personaje <- desplazamiento fin si desplazamiento <- desplazamiento + 1 fin mientras recuperar_de_pila(PC) devolver posicion_personaje fin subrutina
Nombre:mostrar_mapa Recibe: R0 (Direccion de la matriz activa) y R1 (Contador de movimientos) Devuelve: Nada Llama a: pasar_numero_a_ascii	subrutina mostrar_mapa(direccion_matriz_activa, movimientos) guardar_pila(LR) escribir(ESC [2J ESC[H) desplazamiento <- 0 mientras (caracter_actual != '\0') hacer caracter_actual <- leer_memoria(direccion_matriz_activa + desplazamiento) escribir(caracter_actual) desplazamiento <- desplazamiento + 1 fin mientras escribir("Movimientos: ") texto_movimientos <- pasar_numero_a_ascii(movimientos) escribir(texto_movimientos) recuperar_de_pila(PC) fin subrutina

Nombre: pedir_movimiento
Recibe: Nada
Devuelve: R0 (Tecla válida presionada)
Llama a: Ninguna
	subrutina pedir_movimiento()
	guardar_pila(LR)
	
	tecla_valida <- falso
	
	mientras (tecla_valida = falso) hacer
	leer(tecla)
	
	si tecla = 'W' o tecla = 'A' o tecla = 'S' o tecla = 'D' o tecla = 'R' o tecla = 'Q' entonces
	
	tecla_valida <- verdadero
	
	sino si tecla = 'w' o tecla = 'a' o tecla = 's' o tecla = 'd' o tecla = 'r' o tecla = 'q' entonces
	
	// para contar también las minusculas
	tecla_valida <- verdadero
	fin si
	
	fin mientras
	
	recuperar_de_pila(PC)
	devolver tecla
	fin subrutina

Nombre: obtener_posicion_siguiente Recibe: R0 (Tecla de direccion), R1 (Fila	subrutina obtener_posicion_siguiente(tecla, fila_actual, columna_actual, base) guardar_pila(LR)

actual del personaje) R2 (Columna actual) y R3 (Dirección base de la matriz) Devuelve: R0 = Dirección de memoria exacta de la celda destino Llama a: Ninguna	nueva_fila <- fila_actual nueva_columna <- columna_actual si tecla = 'W' entonces nueva_fila <- fila_actual – 1 sino si tecla = 'S' entonces nueva_fila <- fila_actual + 1 sino si tecla = 'A' entonces nueva_columna <- columna_actual – 1 sino si tecla = 'D' entonces nueva_columna <- columna_actual + 1 fin si direccion_destino <- base + (nueva_fila * COLUMNAS + nueva_columna) recuperar_de_pila(PC) devolver direccion_destino fin subrutina
Nombre: verificar_y_mover Recibe: R0 (Dirección destino del personaje), R1 (Dirección actual del personaje), R2 (Tecla presionada para calcular a dónde se empuja la caja) Devuelve: R0 = Verdadero (si se movió) o Falso (si chocó y no se movió) Llama a: obtener_posicion_siguiente (solo si necesita empujar una caja)	subrutina verificar_y_mover(dir_destino, dir_personaje, tecla) guardar_pila(LR) mov_valido <- falso celda_destino <- leer_memoria(dir_destino) si celda_destino = '#' entonces mov_valido <- falso sino si celda_destino = ' ' o celda_destino = '.' entonces escribir_memoria(dir_personaje, elemento_bajo_personaje) elemento_bajo_personaje <- celda_destino si celda_destino = '.' entonces escribir_memoria(dir_destino, '+') sino escribir_memoria(dir_destino, '@') fin si mov_valido <- verdadero sino si celda_destino = '$' o celda_destino = '*' entonces dir_tras_caja <- obtener_posicion_siguiente(tecla, fila_destino, col_destino, base) celda_tras_caja <- leer_memoria(dir_tras_caja) si celda_tras_caja = ' ' o celda_tras_caja = '.' entonces si celda_tras_caja = '.' entonces escribir_memoria(dir_tras_caja, '*') sino escribir_memoria(dir_tras_caja, '$')

	fin si escribir_memoria(dir_personaje, elemento_bajo_personaje) si celda_destino = '*' entonces elemento_bajo_personaje <- '.' escribir_memoria(dir_destino, '+') sino elemento_bajo_personaje <- ' ' escribir_memoria(dir_destino, '@') fin si mov_valido <- verdadero fin si fin si recuperar_de_pila(PC) devolver mov_valido fin subrutina
Nombre: verificar_victoria Recibe: R0 (Dirección de la matriz activa) Devuelve: R0 = Verdadero (si ganó) o falso (si todavía no ganó) Llama a: Ninguna	subrutina verificar_victoria(direccion_matriz) guardar_pila(LR) desplazamiento <- 0 gano <- verdadero mientras (caracter_actual != '\0') hacer caracter_actual <- leer_memoria(direccion_matriz + desplazamiento) si caracter_actual = '$' entonces gano <- falso romper_bucle fin si desplazamiento <- desplazamiento + 1 fin mientras recuperar_de_pila(PC) devolver gano fin subrutina
Nombre: calcular_tiempo_de_juego Recibe: R0 ("inicio" o "final"), R1 = Tiempo inicial (solo si el modo es "final") Devuelve: R0 = Segundos (en inicio) o Registro con Minutos y Segundos (en final) Llama a: Llamada al sistema para obtener la hora	subrutina calcular_tiempo_de_juego(modo, tiempo_inicio) guardar_pila(LR) tiempo_actual <- llamada_sistema_obtener_hora() si modo = "inicio" entonces resultado <- tiempo_actual sino segundos_totales <- tiempo_actual - tiempo_inicio minutos <- 0 mientras (segundos_totales >= 60) hacer segundos_totales <- segundos_totales - 60 minutos <- minutos + 1 fin mientras resultado <- minutos, segundos_totales fin si

	recuperar_de_pila(PC) devolver resultado fin subrutina
Nombre: escribir_ranking y leer_ranking Recibe: R0 (Nombre), R1 (Tiempo total), R2 (Movimientos) Devuelve: Nada Llama a: pasar_numero_a_ascii	subrutina escribir_ranking(nombre, tiempo, movimientos) guardar_pila(LR) archivo <- llamada_sistema_abrir_archivo("ranking_sokoban.txt", modo_agregar) escribir_en_archivo(archivo, nombre) escribir_en_archivo(archivo, " - Tiempo: ") escribir_en_archivo(archivo, tiempo) escribir_en_archivo(archivo, " - Movimientos: ") texto_movimientos <- pasar_numero_a_ascii(movimientos) escribir_en_archivo(archivo, texto_movimientos) escribir_en_archivo(archivo, "\n") llamada_sistema_cerrar_archivo(archivo) recuperar_de_pila(PC) fin subrutina
Nombre: pasar_numero_a_ascii Recibe: R0 (Número entero) Devuelve: R0 (Dirección de memoria del texto convertido a ASCII) Llama a: Ninguna	subrutina pasar_numero_a_ascii(numero) guardar_pila(LR) decenas <- 0 mientras (numero >= 10) hacer numero <- numero - 10 decenas <- decenas + 1 fin mientras unidades <- numero ascii_decenas <- decenas + 48 ascii_unidades <- unidades + 48 guardar_texto(ascii_decenas, ascii_unidades, '\0') recuperar_de_pila(PC) devolver direccion_del_buffer fin subrutina

Rutinas Implementadas

Para esta segunda pre-entrega se implementaron las siguientes subrutinas:

cargar_mapa: Recibe la dirección del mapa original seleccionado en la sección .data, lo copia en la matriz_activa, reinicia el estado de elemento_bajo_personaje a espacio libre ' ' (o meta '.') y obtiene la posición inicial del jugador.

mostrar_mapa: Limpia la pantalla con ANSI Escape y dibuja la matriz en consola carácter por carácter, mostrando debajo el contador de movimientos actualizado.

pasar_numero_a_ascii: Convierte el valor entero del contador de movimientos a sus caracteres ASCII correspondientes para poder imprimirlos en pantalla.

pedir_movimiento: Lee la tecla ingresada por el jugador desde teclado mediante la syscall read, validando que sea una instrucción válida (W, A, S, D para moverse, R para reiniciar y Q para salir).

obtener_posicion_siguiente: Recibe la posición actual del jugador y la tecla presionada, calculando la dirección de la celda destino en la matriz utilizando la dimensión de columnas del mapa activo.

verificar_y_mover: Aplica la lógica del juego. Verifica si la celda destino es una pared, piso, meta, o caja. Si es una caja, calcula la casilla siguiente para validar el empuje. Restaura el piso o meta debajo del jugador y actualiza la matriz.

Link a los archivos

https://github.com/Masimo26/Sokoban-G27-UNGS